

priority_queue 정리

입력 순서와 관계없이 우선순위가 가장 높은 데이터를 먼저 처리하는 C++ 컨테이너 어댑터입니다.

사용처 (Where)

- 최댓값 또는 최솟값을 반복해서 꺼낼 때
- 작업을 우선순위에 따라 처리할 때
- 최단 경로 및 힙 기반 알고리즘을 구현할 때

방법 (How to Use)

- priority_queue 변수 선언
- push()로 데이터 추가
- top()으로 우선순위가 가장 높은 값 확인
- pop()으로 우선순위가 가장 높은 값 제거

기본 정보

- 필요 헤더: #include <queue>
- 기본 선언: priority_queue<int> pq;
- 기본 동작: 큰 값이 먼저 나오는 최대 힙
- 최소 힙: priority_queue<int, vector<int>, greater<int>> pq; (+ <vector>, <functional>)

왜 써야 할까? (Why)

- 최댓값 또는 최솟값을 빠르게 확인
- 삽입-제거 뒤 우선순위가 자동 유지됨
- 힙 자료구조를 직접 구현할 필요가 없음

주요 함수

함수/문법	의미
push(x)	데이터 추가
pop()	우선순위가 가장 높은 데이터 제거
top()	우선순위가 가장 높은 데이터 확인
size()	저장된 데이터 개수
empty()	비어 있는지 확인
emplace(...)	내부에서 객체를 직접 생성

기본 코드 예시

```
#include <iostream>
#include <queue>
using namespace std;

int main() {
    priority_queue<int> pq;
    pq.push(30); pq.push(10); pq.push(50);
    cout << pq.top() << '\n';
    pq.pop();
    cout << pq.top() << '\n';
}
```

priority_queue 비교와 응용

직접 구현이나 일반 queue와 비교하고, 실제 문제에서 쓰는 방식을 확인합니다.

시간 복잡도

동작	복잡도
top()	O(1)
push()	O(log n)
pop()	O(log n)
size(), empty()	O(1)

STL vs 직접 구현 vs C 스타일

구분	STL priority_queue	직접 구현	C 스타일
표현력	우선순위 의도가 분명	힙 연산을 직접 작성	배열 기반 힙 관리
코드 길이	짧고 표준적	상향·하향 이동 필요	인덱스 계산 필요
유지보수	비교 함수만 바꾸기 쉬움	불변식 관리 필요	경계 실수 가능
추천 상황	최대·최소값 반복 처리	특수 힙이 필요할 때	제한된 C 환경

응용: 작은 값부터 출력하기

```
#include <iostream>
#include <queue>
#include <vector>
#include <functional>
using namespace std;

int main() {
    priority_queue<int, vector<int>, greater<int>> pq;
    for (int x : {4, 1, 7, 3}) pq.push(x);
    while (!pq.empty()) {
        cout << pq.top() << ' ';
        pq.pop();
    }
}
```

처음 배울 때 자주 하는 실수

- 기본 priority_queue가 작은 값부터 나온다고 생각함 (기본은 최대 힙)
- 빈 priority_queue에서 top() 또는 pop()을 호출함
- top()으로 확인만 하고 pop()으로 제거하지 않음
- 전체 데이터가 완전히 정렬되어 있다고 생각함
- 같은 우선순위 원소의 입력 순서가 보장된다고 생각함
- 전체 정렬 컨테이너가 아님: 현재 최우선 원소만 top()에 위치하도록 유지함

비슷한 항목과 차이

- queue는 먼저 들어온 데이터를 먼저 처리하는 FIFO 구조
- set은 모든 원소를 정렬 상태로 보관하며 탐색을 지원
- multiset은 정렬 상태를 유지하면서 중복 원소를 허용
- 힙 알고리즘은 vector 등에 힙 연산을 직접 적용할 때 사용
- priority_queue는 top 원소만 직접 확인할 수 있는 컨테이너 어댑터

한 줄 정리: priority_queue는 '최댓값이나 최솟값을 우선 반복 처리할 때' 가장 먼저 떠올리면 좋은 STL입니다.

사용법 숙지를 위한 기본 예제

기본 예제 1. 가장 큰 점수 확인하기

기본 priority_queue는 가장 큰 값이 top()에 위치합니다.

```
priority_queue<int> scores;
scores.push(70);
scores.push(95);
scores.push(80);

cout << scores.top() << '\n';
```

실행 결과: 95

기본 예제 2. 큰 값부터 차례로 꺼내기

top()을 출력한 뒤 pop()하면 다음으로 큰 값이 top()에 옵니다.

```
priority_queue<int> pq;
pq.push(3); pq.push(1); pq.push(5);

while (!pq.empty()) {
    cout << pq.top() << ' ';
    pq.pop();
}
```

실행 결과: 5 3 1